Warm-Started Multigrid for the Particle–Mesh Gravity Solve: Exploiting Temporal Coherence in Distributed Cosmological Simulations

June 21, 2026

Abstract

The gravitational Poisson solve is a central kernel of particle–mesh (PM) cosmological simulations and is conventionally performed with a distributed fast Fourier transform (FFT). At large mesh sizes and high node counts the FFT is increasingly throttled by the global all-to-all communication of its parallel transposes, which scales poorly relative to local computation. We revisit geometric multigrid (MG) as an alternative whose communication is restricted to nearest-neighbor halo exchanges, and we make the observation that, in a time-stepped simulation, the potential is *strongly correlated between successive steps*. Because MG is an *iterative* solver, this temporal coherence can be exploited directly: by *warm-starting* each solve from the previous step's potential, the iteration count required to reach a target accuracy drops from several cycles to one or two. A direct solver such as the FFT cannot exploit this—it recomputes the full transform, and its global communication, every step regardless of how little the solution has changed. We describe the method, including a cosmology-specific growth-scaled predictor that is exact for the linear evolution of the potential, analyze the cost and memory trade-offs, and present measurements in a sharded JAX/jaxdecomp PM implementation. Warm-started multigrid Pareto-dominates a cold multigrid cycle on the accuracy/cost frontier, fits in roughly half the memory of the FFT, and—because evaluating the PM *force* requires the gradient of the potential, which the spectral path obtains via three additional inverse transforms—is faster than the FFT for the full force evaluation at large mesh sizes, by a factor of ~ 1.6 at 1024^3 on 32 GPUs and with the gap projected to widen at higher node counts.

1 Introduction

Particle–mesh (PM) methods [1] and their tree/PM and FastPM descendants [2, 3, 4] underpin a large fraction of cosmological structure-formation modeling. In every variant, each time step requires the solution of the periodic Poisson equation that maps the mass density on a regular mesh to the gravitational potential, from which the force on each particle is obtained as a gradient. The de facto standard for this solve is the fast Fourier transform (FFT): on a single node it is near-optimal, with $\mathcal{O}(N^3 \log N)$ arithmetic for an N^3 mesh and an exact (to round-off) spectral inversion of the Laplacian.

On distributed-memory machines, however, the multidimensional FFT is dominated not by arithmetic but by communication. A 3D transform decomposed over a 2D process grid (a "pencil" decomposition) requires two global transposes, each an all-to-all exchange in which every process communicates with every other. As simulations push to larger meshes and higher node counts to reach the volumes and resolutions demanded by next-generation surveys, this all-to-all becomes the scaling bottleneck: its cost grows with the number of participating processes and saturates

the network bisection bandwidth, while the local arithmetic continues to parallelize cleanly. This motivates revisiting solvers whose communication pattern is *local*.

Geometric multigrid (MG) [5, 6, 7] solves the discretized Poisson equation to discretization-level accuracy in $\mathcal{O}(N^3)$ work using only nearest-neighbor stencil operations and inter-grid transfers. On a sharded mesh these map to halo exchanges between neighboring subdomains—point-to-point communication that scales far more gently than an all-to-all. As a stand-alone single Poisson solve on a fast modern interconnect, MG is nonetheless typically *slower* than the FFT: the spectral inversion is exact and hard to beat per floating-point operation, and the FFT's all-to-all, while global, is highly optimized.

The central observation of this note is that the PM context provides additional structure that an iterative solver can exploit but a direct solver cannot. Over a simulation time step the density field, and hence the potential, evolves *slowly and predictably*: the large-scale modes follow linear growth, and only the nonlinear, mode-coupling part changes appreciably. An iterative solver can be *warm-started* from the previous step's potential, so that it need only correct this small change rather than reconstruct the solution from scratch. The FFT, being a direct solver, has no mechanism to incorporate a prior estimate; it pays its full cost, including its global communication, on every step. We show that warm-starting closes—and for the full force evaluation reverses—the per-solve gap, turning the temporal coherence of the simulation into a concrete performance advantage for multigrid.

Contributions. (i) We formulate warm-started multigrid for the PM gravity solve and introduce a cosmology-specific *growth-scaled predictor* that, by linearity of the Poisson operator, exactly removes the linear-growth component of the inter-step change (§3). (ii) We give a cost, communication, and memory analysis that explains why the advantage is largest for the full force evaluation and at scale (§4). (iii) We report an implementation in a sharded JAX/JAXDECOMP PM code and measurements of the accuracy/cost trade-off, full-step timings, and field-level comparisons (§6), together with practical guidance (§7).

2 Background

2.1 The PM gravity solve

On a periodic cubic mesh of side N with spacing h, particles are deposited (e.g. by cloud-in-cell interpolation) to form the density contrast $\delta(\mathbf{x})$. The (comoving, suitably normalized) potential satisfies

$$\nabla^2 \varphi = \delta, \tag{1}$$

and the force per unit mass is $\mathbf{g} = -\nabla\varphi$. Discretizing the Laplacian with the standard second-order seven-point stencil yields the linear system

$$A\,\varphi = \delta, \tag{2}$$

where A is symmetric negative-semidefinite with a one-dimensional null space spanned by the constant mode (the discrete analogue of the requirement that a periodic source integrate to zero). Solvability requires $\sum_{\mathbf{x}} \delta(\mathbf{x}) = 0$; the FFT enforces this implicitly by setting the $\mathbf{k} = 0$ Fourier coefficient of the potential to zero, and in the multigrid solver we project the mean out of δ before solving.

2.2 Spectral (FFT) solution and its communication cost

The FFT solves (2) directly,

$$\varphi = \mathcal{F}^{-1}\Big[\hat{\delta}(\mathbf{k}) \,/\, \lambda(\mathbf{k})\Big], \tag{3}$$

where $\lambda(\mathbf{k})$ are the eigenvalues of A (for the discrete stencil, $\lambda = -\frac{4}{h^2}\sum_i \sin^2(k_i h/2)$; the continuum choice $\lambda = -|\mathbf{k}|^2$ is also common). The cost is fixed: a forward transform of δ, a pointwise multiply, and an inverse transform. The force additionally requires the gradient; in the spectral representation this is a pointwise multiply by $i\,\widehat{\partial}_i$ followed by an inverse transform per Cartesian component, so a full force evaluation costs *four* transforms in total (one forward, three inverse). On a distributed mesh each transform entails one or two global transposes, i.e. all-to-all communication, which dominates the wall-clock time at scale and is independent of any prior knowledge of the solution.

2.3 Geometric multigrid

Multigrid solves (2) iteratively by combining a local *smoother*, which efficiently damps high-frequency error components, with a coarse-grid correction that handles the smooth, long-wavelength error [5, 6, 7]. A single V-cycle on a hierarchy of meshes performs, at each level: ν_1 pre-smoothing sweeps (e.g. damped Jacobi); restriction of the residual to the next-coarser mesh; a recursive solve of the residual equation; prolongation of the correction back to the fine mesh; and ν_2 post-smoothing sweeps. Each operation is a local stencil or inter-grid transfer. For the model Poisson problem the V-cycle reduces the error by a mesh-independent factor $\rho \in (0, 1)$ per cycle,

$$\left\|\varphi^{(k)} - \varphi^{\star}\right\| \;\le\; \rho^{k}\, \left\|\varphi^{(0)} - \varphi^{\star}\right\|, \tag{4}$$

with ρ typically of order 0.1 for a well-tuned cycle, giving the optimal $\mathcal{O}(N^3)$ complexity. Crucially, (4) shows that the number of cycles needed to reach a target tolerance depends on the quality of the initial guess $\varphi^{(0)}$: the work is set by the ratio of initial to final error, not by the solution itself.

On a sharded mesh, the smoother and transfer stencils require only the values in a thin boundary layer of each subdomain. These are supplied by *halo exchanges*: point-to-point messages between geometrically neighboring processes whose volume is the surface area of the subdomain. This nearest-neighbor pattern is the structural advantage of multigrid over the FFT's all-to-all on large process grids. (The coarsest levels, where a subdomain holds only a few cells, are handled by gathering the small coarse grid onto a replicated copy and solving it redundantly without further inter-node communication; see §5.)

3 Warm-started multigrid

3.1 Temporal coherence and the initial guess

A direct solver returns $\varphi^{\star}$ regardless of input; an iterative solver returns $\varphi^{\star}$ to a tolerance after a number of steps that decreases as $\varphi^{(0)}$ improves. In a time-stepped PM simulation, the potential at step n differs only slightly from that at step $n-1$, because the density evolves continuously and the time step is bounded by accuracy and stability of the integrator. The previous potential is therefore an excellent initial guess. If the relative inter-step change is $\epsilon = \|\varphi_n - \varphi_{n-1}\| \,/\, \|\varphi_n\|$, then from the cold start $\varphi^{(0)} = 0$ the initial relative error is unity and reaching tolerance τ needs $k_{\mathrm{cold}} \approx \log\tau / \log\rho$ cycles, whereas from the recycled start the initial error is $\mathcal{O}(\epsilon)$ and

$$k_{\mathrm{warm}} \;\approx\; \frac{\log(\tau/\epsilon)}{\log\rho} \;=\; k_{\mathrm{cold}} - \frac{\log(1/\epsilon)}{\log(1/\rho)}. \tag{5}$$

For ϵ of a few percent and $\rho \sim 0.1$ this removes one to two cycles, i.e. most of the work of a cold solve.

3.2 Predictors

We consider three initial guesses of increasing sophistication.

(i) Recycling. $\varphi_n^{(0)} = \varphi_{n-1}$. Simple, requires no extra storage beyond the previous potential, and already captures the bulk of the solution.

(ii) Growth-scaled predictor. The Poisson operator (2) is linear, so a uniform rescaling of the source rescales the solution identically. In the linear regime the density contrast grows as the linear growth factor $D(a)$, $\delta \to (D_n/D_{n-1})\,\delta$, and hence

$$\varphi_n^{(0)} = \frac{D(a_n)}{D(a_{n-1})}\,\varphi_{n-1} \tag{6}$$

exactly removes the linear-growth component of the inter-step change, leaving only the nonlinear, mode-coupling residual for the cycles to resolve. This predictor is specific to cosmological growth and costs only a scalar multiply.

(iii) Polynomial extrapolation. With more history, a linear (or quadratic) extrapolation $\varphi_n^{(0)} = 2\varphi_{n-1} - \varphi_{n-2}$ ($3\varphi_{n-1} - 3\varphi_{n-2} + \varphi_{n-3}$) captures steady drift of the potential and can further reduce ϵ when the trajectory is smooth.

3.3 Defect-correction equivalence and Krylov acceleration

Warm-starting is mathematically a defect correction: writing $\varphi = \varphi^{(0)} + e$, the error satisfies $A\,e = \delta - A\varphi^{(0)} \equiv r^{(0)}$, the residual of the initial guess. Because $\varphi^{(0)}$ is accurate, $r^{(0)}$ is small, and the multigrid cycles converge it rapidly; one recovers $\varphi = \varphi^{(0)} + e$. This framing makes explicit that the solver operates on a small quantity, which is also numerically favorable. When a guaranteed monotone decrease of the residual is desired (e.g. for tight tolerances or stiff configurations), the same $\varphi^{(0)}$ can seed a multigrid-preconditioned conjugate-gradient iteration, with multigrid as the preconditioner and the recycled potential as the initial iterate.

3.4 Error propagation across steps

A potential concern with recycling is the accumulation of solver error along the trajectory: each step's force is computed from an approximate potential, perturbing the particle positions and hence the next step's source. In practice the scheme is self-correcting. The integrator's own truncation error sets the accuracy floor; provided each warm solve reduces the residual below this floor (a small, fixed number of cycles), the per-step solver error does not accumulate coherently, and the final field tracks the reference (FFT) solution to a controlled tolerance set by the chosen cycle count (§6). The first step has no predecessor and is initialized once with an accurate solve (a deep multigrid cycle, or a single FFT solve used only at initialization).

3.5 Why direct solvers cannot exploit temporal coherence

The FFT has no notion of an initial guess: (3) is evaluated in full on every step, including its global transposes, whether the potential has changed by a part in 10^3 or by order unity. Temporal coherence is therefore a resource available exclusively to iterative solvers, and it is precisely aligned with the slowly varying character of the PM potential. This is the mechanism by which warm-started multigrid overcomes the per-solve disadvantage that multigrid would otherwise carry against a highly optimized spectral solver.

4 Cost, communication, and memory

4.1 Operation and communication counts

Per force evaluation, the spectral path performs four global transforms (one forward, three inverse for the gradient components), i.e. of order eight global transposes. The multigrid path performs one warm solve—a few V-cycles, each dominated by halo exchanges proportional to subdomain surface area—followed by a real-space finite-difference gradient that reuses the same local halo machinery and requires *no* global communication. Thus the asymptotic communication advantage of multigrid is twofold: it replaces all-to-all transposes by nearest-neighbor exchanges, and it obtains the gradient locally rather than by three additional inverse transforms. The first factor grows with node count; the second is a fixed $\sim 4\times$ reduction in transform-equivalent work that applies even at modest scale.

4.2 Memory

The spectral solve operates in complex arithmetic and materializes transposed copies of the field during the transform, so its transient footprint is several times the real mesh. The multigrid solve operates entirely in single-precision real arithmetic on the mesh hierarchy (whose total size is $\frac{8}{7}N^3$). In our sharded implementation this lets multigrid run within roughly half the device memory of the FFT path; empirically, configurations that exhaust memory for the FFT remain feasible for multigrid at the same node count.

4.3 Scaling expectations

At fixed mesh size, increasing the node count shrinks each subdomain: the FFT all-to-all involves more participants and smaller messages (latency-bound, scaling unfavorably), whereas multigrid's halo exchanges remain nearest-neighbor and its per-node arithmetic falls. The warm-start advantage is therefore expected to grow with node count, and—because the gradient cost is incurred every step regardless of mesh size—to be present already at moderate scale for the full force evaluation.

5 Implementation

We implement both solvers in JAX [8] with distributed transforms and halo exchanges provided by JAXDECOMP [9], within the JAXPM particle–mesh framework [10]. The mesh is sharded over a two-dimensional device grid; the third axis is local. Several implementation choices matter for performance:

- **Whole-program compilation.** Staging the entire V-cycle into a single compiled executable, rather than dispatching each kernel eagerly, is essential; in our tests it accounted for a 5–6× reduction in multigrid run time.
- **Communication-avoiding smoothing.** A damped-Jacobi smoother is applied with a widened halo so that several sweeps follow a single exchange, reducing the number of halo communications; the relaxation factor is tuned to the three-dimensional stencil.
- **Coarse-grid agglomeration.** Once a coarse level is small enough that each subdomain holds only a few cells, the grid is gathered to a replicated copy and the remaining levels are solved redundantly on each device, eliminating latency-bound exchanges on the deepest levels and restoring the long-wavelength accuracy that heavy sharding would otherwise truncate.
- **Warm-start threading.** Because multi-stage and adaptive ODE integrators evaluate the force several times per step (and on rejected steps), the recycled potential is threaded through an explicit fixed-step leapfrog—one force evaluation per step—which is the conventional PM integrator and makes the "previous potential" unambiguous.
- **Mass assignment halo.** Cloud-in-cell deposition on a sharded mesh must exchange a halo wide enough to capture particle displacements that cross subdomain boundaries; too narrow a halo leaves shard-boundary artifacts in the painted field. The halo width must therefore scale with the typical displacement in cells, i.e. with mesh resolution.

6 Results

We evaluate on NVIDIA A100 GPUs. All comparisons use identical initial conditions for each solver, with the multigrid solver configured to the same discrete operator as the spectral reference so that differences reflect solver convergence rather than discretization. Accuracy is quoted as the relative L_2 difference of the final density field against the FFT solution.

6.1 Accuracy/cost trade-off

Table 1 reports a full 256^3 PM run of 30 steps as the number of warm-start V-cycles is varied. Each warm cycle reduces the final-field error by roughly an order of magnitude, and the warm scheme *Pareto-dominates* a cold cycle: at equal cost (three cycles) warm-start reaches 3.7×10^{-3} where a cold cycle reaches only 3.4×10^{-2}, and two warm cycles match the cold three-cycle accuracy at lower cost. A single warm cycle is the fastest configuration but is generally too coarse for production accuracy.

6.2 Full-step timing at scale

Table 2 reports the full force-evaluation cost per step across mesh sizes and node counts. On a single GPU and a few GPUs the optimized FFT remains fastest for a single solve, but the gap narrows with scale; at 1024^3 on 32 GPUs the warm-started multigrid step is $\sim 1.6\times$ faster than the FFT step. The crossover is driven by the gradient cost (four transforms for the FFT versus one local finite difference for multigrid) compounded by the growing relative cost of the all-to-all at higher node counts. We additionally observe that the multigrid path runs within roughly half the device memory of the FFT path, remaining feasible at node counts where the FFT exhausts memory.

configuration	cost (ms/step)	final-field rel. L_2
FFT (reference)	17.9	—
cold MG (3 cycles)	23.3	3.4×10^{-2}
warm MG, 1 cycle	16.5	2.7×10^{-1}
warm MG, 2 cycles	20.1	3.0×10^{-2}
warm MG, 3 cycles	23.3	3.7×10^{-3}
warm MG, 4 cycles	26.6	5.2×10^{-4}

Table 1: Accuracy/speed trade-off versus warm-cycle count, 256^3 mesh, 30 steps, single GPU, using the growth-scaled predictor (6). Warm-start dominates the cold cycle: same cost, roughly an order of magnitude better accuracy.

mesh / GPUs	FFT (ms/step)	warm MG (ms/step)	speedup
256^3 / 1	17.9	20.1	0.89×
512^3 / 4	99.2	131	0.76×
1024^3 / 32	1611	980	**1.64**×

Table 2: Full PM force-evaluation cost per step (paint, solve, gradient, read). Warm multigrid uses the growth-scaled predictor and 2–3 cycles; the FFT uses the matched discrete operator. The advantage appears and grows with scale.

6.3 Field-level comparison

At the field level the warm-started multigrid and FFT runs are visually indistinguishable; the residual is a few percent in relative L_2 and is localized to the dense cores of collapsed halos, where the discrete operator and the finite cycle budget differ most. The cosmic-web structure— filaments, walls, and voids—is reproduced faithfully. (Care is required in the mass-assignment halo width: at fixed box size, increasing the mesh resolution increases particle displacements measured in cells, so an under-sized halo produces shard-boundary artifacts in the deposited field; these are a property of the parallel deposition, common to both solvers, and are removed by sizing the halo to the displacement scale.)

7 Discussion

The results support a simple operational recipe. Initialize the potential once with an accurate solve. On each subsequent step, form the initial guess from the growth-scaled previous potential (6) (or a polynomial extrapolation) and apply a small fixed number of V-cycles, recycling the result; two to three cycles is typically the sweet spot, chosen from the trade-off in Table 1 according to the force accuracy the integrator requires.

Several limitations and extensions are worth noting. First, the per-solve comparison still favors the FFT; the multigrid advantage is realized specifically through temporal warm-starting and the local gradient, and is therefore tied to the time-stepped force-evaluation context rather than to an isolated Poisson solve. Second, the crossover scale depends on the interconnect: on networks where the all-to-all is comparatively fast, larger meshes or higher node counts are required before multigrid wins on the bare solve, although the gradient-cost factor applies throughout. Third, the accuracy is tunable but not exact; applications requiring spectral accuracy of the potential itself (as

opposed to the force) may prefer the FFT or a Krylov-accelerated multigrid to a tight tolerance. Finally, the same temporal-coherence argument applies to other slowly varying mesh quantities and to adaptive or multi-rate time stepping, where the predictor can be matched to the local step size.

8 Conclusions

We have shown that the temporal coherence of the gravitational potential in particle–mesh simulations is a directly exploitable resource for an iterative Poisson solver, and have realized it as warm-started geometric multigrid with a cosmology-specific growth-scaled predictor. Warm-starting reduces the solve to one or two multigrid cycles, Pareto-dominates a cold cycle on the accuracy/cost frontier, and—because the PM force requires the gradient of the potential, obtained locally by multigrid but via three additional inverse transforms by the FFT—makes multigrid faster than the distributed FFT for the full force evaluation at large mesh sizes, with a memory footprint roughly half that of the spectral path and an advantage projected to grow with node count. These properties make warm-started multigrid an attractive gravity solver for large, communication-bound distributed PM simulations.

References

[1] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*, Adam Hilger, Bristol, 1988.

[2] J. S. Bagla, "TreePM: A Code for Cosmological N-Body Simulations," *Journal of Astrophysics and Astronomy* **23**, 185 (2002).

[3] V. Springel, "The cosmological simulation code GADGET-2," *Mon. Not. R. Astron. Soc.* **364**, 1105 (2005).

[4] Y. Feng, M.-Y. Chu, U. Seljak, and P. McDonald, "FastPM: a new scheme for fast simulations of dark matter and haloes," *Mon. Not. R. Astron. Soc.* **463**, 2273 (2016).

[5] A. Brandt, "Multi-Level Adaptive Solutions to Boundary-Value Problems," *Mathematics of Computation* **31**, 333 (1977).

[6] W. L. Briggs, V. E. Henson, and S. F. McCormick, *A Multigrid Tutorial*, 2nd ed., SIAM, Philadelphia, 2000.

[7] U. Trottenberg, C. W. Oosterlee, and A. Schüller, *Multigrid*, Academic Press, London, 2001.

[8] J. Bradbury *et al.*, "JAX: composable transformations of Python+NumPy programs," `http://github.com/google/jax` (2018).

[9] JAXDECOMP: JAX bindings for distributed domain decomposition and FFTs, `https://github.com/DifferentiableUniverseInitiative/jaxDecomp`.

[10] JAXPM: a differentiable particle–mesh library in JAX, `https://github.com/DifferentiableUniverseInitiative/JaxPM`.